Explainable Region-Based Routing Across Heterogeneous Language Models with Growing Neural Gas

Ujjwal Sharma[1⋆], Pranav Iyer[1], Ishaa Bhalekar[1], and Siya Jain[1]

Computer Science Engineering (Data Science), NMIMS, Mumbai, India
ujjwal.sharma04@nmims.in, pranav.iyer05@nmims.in,
ishaasanjay.bhalekar38@nmims.in, siya.jain20@nmims.in

Abstract. Language models vastly differ in their price and capability, creating a perceived trade-off. this paper addresses this false dichotomy by proposing a novel routing framework that facilitates cost-saving without sacrificing accuracy. Existing routers are either static clustering schemes with limited adaptability or opaque neural classifiers that offer no insight into their routing behavior - we propose a novel framework that leverages a Growing Neural Gas (GNG) topology to address their shortcomings. The framework learns a graph of prototype vectors over prompt embedding space, where each node records the historical error rates of all candidate models based on the prompts mapped to it. A decision is then explained by one region of embedding space, and a single scalar trades cost against quality. Each node evaluates locally, and the design permits a change of model pool without retraining from scratch, an extension we hope further research can explore. We evaluated our work on 35,189 RouterBench prompts and 11 large language models (LLMs), with every router on the same embeddings and split: the shared-topology variant is competitive with k-nearest-neighbor (KNN), support vector machine (SVM), multilayer perceptron (MLP), and UniRoute on deferral-curve metrics. Its area is 0.785, 0.0035 above UniRoute (paired bootstrap interval 0.0015 to 0.0056), and it reaches GPT-4's accuracy at 72% of GPT-4's normalized cost on the test split. The margins over UniRoute are small. The GNG router is, however, a competitive method that is very explainable.

Keywords: LLM routing · Neural gas · Mixture-of-experts · Model selection · Explainable AI · Cost-aware inference

1 Introduction

When it comes to Large language models (LLMs), generalist models do not neccessarily outperform a specialist models in tasks that fall under the latter's purview. This indicates that a model's performance is inherently task-dependent, with no single model guaranteed to perform optimally across all tasks. Wolpert

⋆ Corresponding author.

and Macready's No Free Lunch theorems state the general version of this point for optimization algorithms [37].

Querying each and every model to arrive at the best possible answer, while logically sound, implies incurring inference costs for all those models and scales poorly with model pool size. This limitation motivates LLM Routing: a distinct research subfield in which each prompt is directed to a single model deemed most suited to the task at hand [24, 34].

Cascades and cost-aware selection were the first approaches to attempt to build this towards a more useful routing strategy [4, 6]. More recent routing frameworks, such as RouteLLM, learn from preference data [26], or cluster the prompts (UniRoute [17]), or embed the models themselves [40], or use graphs [8], or training with a cost-aware contrastive loss [31].

One major drawback to some of these approaches is that most accurate routers cannot reason why they chose a model. This makes them hard to debug, improve and audit, which is crucial to deciding the bill and the quality of services for customers of these systems [29]. There also exist some results suggesting the complexity may not add much: a plain k-nearest-neighbor (KNN) router can match or beat learned deep learning ones [21], and in mixture-of-experts models the routing topology does not determine final quality [33]. We therefore hoped to build a router that provides at least comparable results with these methods and that which provides some understanding to why a particular model was chosen.

We design it with Growing Neural Gas (GNG) [9], using a graph of prototype vectors that grows to cover a data distribution over a set model space. We grow a GNG over prompt embeddings and store the error rate of every candidate model in each node for all the prompts corresponding to the node. To route a prompt we find its nearest node (embedded prompt) and pick the model with the best cost-adjusted score there. The node and the neighborhood values provide a tangible explanation to why a particular model was selected. Our paper evaluates this as a static router on a fixed pool owing to cost constraints.

A major infuence of building a router like this is potential for applications in networking distribution based on model wants for a distributed multi-agent using system.

Pools do not stay fixed, and that shaped the design. One of the primary future visions of the authors for this research area is to explore evolving adaptive pools. Models can added and retired at rapid pace in modern systems, and if a router must be retrained or reclustered after each change, overhead may be expensive and need workarounds or raise future costs. Since all statistics are local to nodes, use per-model cost constants that need no refitting, and unused nodes age out using a decay function (an addition we made to standard GNG).

These choices should make adding or removing a model a local operation, and Section 4.4 describes the standard we propose. We treat it as a design extension and leave its evaluation to future work.

Our contributions are:

- An explainable, region-based router: a GNG over prompt embeddings with per-node, per-model error rates and one Lagrangian parameter for the cost-quality trade-off.
- Two variants. *GNG-shared* keeps one topology with statistics for all models. *GNG-isolated* keeps one topology per model, grown on the prompts that model answers correctly.
- An evaluation on RouterBench against KNN, support vector machine (SVM), multilayer perceptron (MLP), and UniRoute, with all routers on the same embeddings and split, bootstrap intervals for the differences, and the variation of GNG training across five seeds.
- A design extension for changing pools, built on node-local statistics and aging, with procedures for adding and removing models.

Section 2 covers related work and Section 3 states the problem. Section 4 describes the method, Sections 5 and 6 give the experiments, and the remaining sections discuss the results, limitations, and future work.

2 Related Work

2.1 Mixture of Experts

Mixture-of-experts (MoE) models send an input to specialized sub-models according to its position in a learned space, with a gating network assigning soft weights to local experts [16]; [25] surveys recent work. Sparsely gated and switch-style layers made MoE practical at scale by running only a few experts per token [7, 30]. Hash Layers swap the learned gate for fixed hash functions to cut routing overhead [28]. Other work changes the routing geometry, for example curvature-guided routing for graphs [2] and mixtures of curvature experts for hyperbolic LLMs [13]. These systems train the experts and the gate together. Our router sits outside the models and treats independently trained LLMs as black boxes, as RouteLLM does [26].

2.2 Routing Across Independently Trained LLMs

Routers for separate LLMs mostly fall into two groups. Cluster-then-assign methods group queries and give each group its best model [17, 32, 40]. Performance-prediction methods estimate how well each model will do on a query and take the arg-max [8, 26, 31]. Other directions include multi-round routing by reinforcement learning [38], routing from prefill activations [35], deciding when to invoke reasoning [36], scaling expert models through unsupervised domain discovery [12], characterizing expert-router performance [27], and routing for LLM-as-a-judge [19,39]. RouterBench [14] and LLMRouterBench [20] offer shared benchmarks, and two surveys organize the area [24,34]. Cluster-based routers are our closest relatives because they too attach per-model statistics to regions of embedding space. They fix the number of regions in advance, for example with k-means. A GNG lets the data decide how many regions there are and where they sit, up to a node limit.

2.3 Explainable Routing

IRT-Router describes model ability and query difficulty in a latent space based on selection of test items using psychological evaluation [32]. The parameters are interpretable, at the price of an assumption. For high-stakes decisions, models that are interpretable by design are generally preferable to explanations added after the fact [29]. Our router gives a partial explanation of this kind: each decision maps to a bounded region with stored statistics, albeit not psychologically driven in design.

2.4 Incremental Topology Learning and Drift

Self-organizing maps [18] and neural gas [22] learn prototype vectors, and competitive Hebbian learning produces topology-representing networks [23]. GNG adds nodes and edges to a stream of data wherever the current graph represents it poorly. Edges carry an age that resets when the edge is used, and old edges are removed [9]. A later variant tracks non-stationary distributions [10], and we use the same idea for stale regions. Adaptive Resonance Theory (ART) decides with a vigilance parameter whether an input joins an existing category or starts a new one [3]. Work on concept drift motivates adapting without a full retrain [11]. As far as we know, neither GNG nor ART has been used to route over a changing pool of LLMs.

3 Problem Formulation

Let $\mathcal{M} = \{m_1, \ldots, m_K\}$ represent a pool of LLMs while $e(x) \in \mathbb{R}^d$ represents the embedding of prompt x under a fixed encoder. A model m_k earns a quality score $q_k(x) \in [0, 1]$ (answer correctness in our experiments) at cost $c_k(x) \geq 0$. A router $r : \mathbb{R}^d \to \mathcal{M}$ then selects one model per prompt. We want

$$\max_r \ \mathbb{E}_x\big[q_{r(x)}(x)\big] \quad \text{s.t.} \quad \mathbb{E}_x\big[c_{r(x)}(x)\big] \leq B \tag{1}$$

for a budget B, with $r(x)$ explainable and, as a design goal, able to survive changes in $\mathcal{M}$. We state three requirements.

- R1 **Cost-quality control.** One scalar should trace the accuracy-cost frontier, a deferral curve, so an operator can choose B after training.
- R2 **Auditability.** Each decision should trace to a small, inspectable part of the learned structure.
- R3 **Pool adaptivity.** Adding or retiring a model should not involve full model retraining, instead relying on incremental changes.

Training data constitutes of prompts along with precomputed scores and costs for every model - the router never calls a candidate model during inference. The experiments test R1 and R2. R3 remains a design requirement that informs the method (Section 4.4) but is not evaluated.

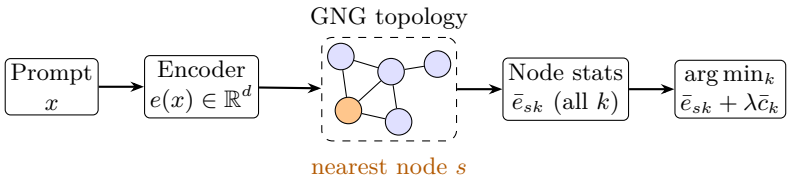


Fig. 1. Routing pipeline (GNG-shared). A prompt is embedded and mapped to its nearest GNG node s (highlighted), and the model that minimizes the cost-penalized error of the node's stored statistics is selected. The node, its neighbors, and its statistics form the explanation.

4 Method

The router is built in two stages (Fig. 1). First, a Growing Neural Gas (GNG) network is constructed over the training prompt embeddings, partitioning the embedding space into regions of similar prompts. Each node is then assigned the observed error rates of candidate models on the training prompts associated with it. During inference, an incoming prompt is mapped to its nearest node, and the model with the best cost-adjusted score for that node is selected.

4.1 Growing the Topology

The graph $G = (V, E)$ has nodes i with a reference vector $w_i \in \mathbb{R}^d$; an accumulated error E_i; and a time of last activation (which allows for edge aging). We start with two training embeddings and present the training prompts in random order for T passes. For an input $e = e(x)$:

1. Find the nearest node s_1 and the second-nearest node s_2 using squared Euclidean distance. Let $d_1 = \|e - w_{s_1}\|^2$.
2. Add d_1 to E_{s_1} and $\sqrt{d_1}$ to a global distance counter D.
3. Move the winner and its topological neighbors toward the input: $w_{s_1} \leftarrow w_{s_1} + \epsilon_b(e - w_{s_1})$ and $w_j \leftarrow w_j + \epsilon_n(e - w_j)$ for $j \in \mathcal{N}(s_1)$, with $\epsilon_b > \epsilon_n$.
4. Increase the age of every edge at s_1. Create or reset the edge (s_1, s_2) with age 0, but only if $\|w_{s_1} - w_{s_2}\| \leq \rho$ (or if the graph has fewer than 20 nodes).
5. Remove edges older than $a_{\max}$, and delete nodes left without edges.
6. If $D \geq \theta$ and $|V| < N_{\max}$, reset D to 0 and insert a node u halfway between the node $q^\star = \arg\max_i E_i$ and its highest-error neighbor f, so $w_u = \frac{1}{2}(w_{q^\star} + w_f)$. Replace the edge $(q^\star, f)$ with $(q^\star, u)$ and (u, f), connect u to every node within distance ρ, multiply $E_{q^\star}$ and E_f by $\alpha < 1$, and set $E_u = E_{q^\star}$.
7. Multiply all node errors by $\gamma < 1$.

After training, we remove nodes that were not activated in the last τ steps. This is in line with standard GNG [9], however two changes are implemented: insertion is triggered based on a threshold θ rather than at intervals, and edge creation is constrained within a maximum length. The growth makes use of quantization

Algorithm 1 GNG router training (GNG-shared)

Input: training embeddings $\{e(x_n)\}$, correctness $q_k(x_n)$ for all k
1: Initialize two nodes from the first two training embeddings
2: **for** pass $= 1, \ldots, T$ **do**
3: **for** each training prompt x in random order, step t **do**
4: Apply the GNG update (steps 1–7 above)
5: **end for**
6: **end for**
7: Remove nodes inactive for more than τ steps
8: Assign every training prompt to its nearest node
9: **for** each node i and model k **do**
10: $\bar{e}_{ik} \leftarrow$ mean of $1 - q_k(x)$ over prompts assigned to i
11: **if** node i has no assigned prompts **then** $\bar{e}_{ik} \leftarrow 1 - \bar{q}_k$ (global mean)
12: **end for**
13: $\bar{c}_k \leftarrow$ mean cost of model k (one constant per model)

error, making the topology unsupervised. Performance data is used only in the second stage of the process. The implementation can also allow scaling each update by a temporal weight $\exp(-\kappa t)$. This paper sets $\kappa = 0$ for both variants for ease of comparison; Section 5 reports what happens otherwise. Table 1 gives the parameter values.

Initialization, winner selection, growth, and error estimation. The graph is initialized with two nodes whose reference vectors are the first two training embeddings. For every presented prompt the winner s_1 is the node with the smallest squared Euclidean distance to $e(x)$, and s_2 is the next closest. Because the embeddings have unit norm (Section 5), this ordering is identical to the ordering by cosine distance. New nodes are added only when the accumulated winner distance D reaches θ, halfway between the node with the largest accumulated error and its highest-error neighbor, and the number of nodes is bounded by $N_{\max}$. The final number of nodes is therefore a result of training and not a setting (107 nodes for GNG-shared, and 105 to 111 across five seeds). Model-specific error estimation happens only after the topology is fixed. Every training prompt is assigned to its nearest node, and for each node i and model k the error rate $\bar{e}_{ik}$ is the mean of $1 - q_k(x)$ over the prompts assigned to i (lines 8–12 of Algorithm 1). A node that receives no training prompts uses the global training error rate of the model. These estimates do not affect where nodes are placed. In GNG-isolated the same estimation is applied separately to the topology of each model.

4.2 Cost-Aware Routing Rule

Each prompt provided to the system must be embedded and the nearest node is found. We choose

$$r(x) = \arg\min_k \ \bar{e}_{sk} + \lambda \bar{c}_k, \tag{2}$$

where $\bar{e}_{sk} = 1 - \bar{q}_{sk}$ is the error rate of model k at node s, $\bar{c}_k$ is the mean cost per query at k and $\lambda \geq 0$ sets the cost sensitivity. This is the Lagrangian relaxation of

the objective function described before, applied to each region turn-based. With $\lambda = 0$ the router takes the most accurate model in the region, and as λ increases, it increases the emphasis on including cost into the decision. Sweeping λ gives the deferral curve of accuracy against cost that we use for evaluation, which satisfies R1. Cost enters through per-model constants and not per-node estimates, which is also how the KNN, SVM, MLP, and UniRoute baselines use it. A Lagrangian sweep reaches only the points on the convex hull of the per-region quality-cost pairs, and budgets in between come from interpolating along the sweep.

Selecting the trade-off parameter. The parameter λ is not tuned during training, and the node statistics do not depend on it. It is chosen after training as an operating point on the deferral curve: for a given budget B, or a minimum accuracy requirement, an operator picks the value of λ whose point on the curve meets that budget or accuracy, ideally using held-out prompts. In the experiments we do not select a single λ. We sweep 501 values (0 and 500 log-spaced values in $[10^{-1}, 10^{6}]$, Section 5), and Area, Area50, and QNC summarize the behavior across the whole cost range, across its lower half, and at the accuracy requirement of matching the best single model, respectively. Because prices enter only through the per-model constants $\bar{c}_k$, a change in model prices changes the arg-min in Eq. (2) without refitting any node statistic. The sensitivity of the results to different price levels was not evaluated.

4.3 Variants

GNG-shared grows one topology on all training embeddings (Algorithm 1) and stores the error rates of all K models in every node. Every model is scored in the same regions, so the comparisons in Eq. (2) are like for like.

GNG-isolated grows a separate topology G_k for each model m_k, using only the training prompts that m_k answers correctly, so G_k describes where m_k is competent. Each node of G_k stores the error rate of m_k over all training prompts for which it is the nearest node in G_k. A prompt far from every node of G_k lies outside the competence region of m_k, and we penalize this softly. Let s_k be the node of G_k nearest to $e(x)$, let $d_k = \|e(x) - w_{s_k}\|$, and let ρ_k be the 90th percentile of the distances from the correct training prompts of m_k to G_k. The rule becomes

$$r(x) = \arg\min_k \; \bar{e}^{(k)}_{s_k} + \beta \max(0, d_k - \rho_k) + \lambda \bar{c}_k, \tag{3}$$

where β weights the boundary penalty (not cost).

4.4 Design Extension: Changing Pools

Several choices above were made with changing pools in mind for future study. Statistics sit in the nodes, so a new or updated model affects only the regions where it is evaluated. Because cost enters only through one constant per model, nothing else must be changed. Edges that are not reinforced age out, nodes left without edges are deleted, and nodes inactive for more than τ steps are

Algorithm 2 Adding a model without regrowing the topology

Input: trained graph G; calibration prompts S with scores and costs of the new model m_{K+1}

for each $x \in S$ **do**
    $s \leftarrow$ nearest node to $e(x)$; accumulate $1 - q_{K+1}(x)$ at s
end for
$\bar{e}_{s,K+1} \leftarrow$ mean accumulated error at node s; for nodes with fewer than $n_{\min}$ calibration prompts, fall back to the global mean error of m_{K+1}
$\bar{c}_{K+1} \leftarrow$ mean cost of m_{K+1} over S
Optionally continue Algorithm 1 on S to refine regions where m_{K+1} changes the preferred model

removed [10]. Algorithm 2 adds a model m_{K+1} to GNG-shared. Removing a model deletes its statistics entry from every node, an $O(|V|)$ operation, because Eq. (2) recomputes the arg-min at query time. A prototype command-line tool implements removal (for GNG-isolated, by deleting that model's topology) and a "refresh" step that prunes stale nodes and recomputes error rates on new data. The experiments in this paper use a fixed pool and do not run Algorithm 2.

Complexity. Let $N = |V|$. A training step costs $O(Nd + |E|)$ for nearest-node search and graph bookkeeping, and an insertion costs $O(N)$. The statistics pass over n training prompts costs $O(n(Nd + K))$, and memory is $O(N(d + K))$. Inference with linear search costs $O(Nd)$ for GNG-shared and $O(\sum_k N_k d)$ for GNG-isolated. Adding a model to GNG-shared costs $O(|S|\, Nd)$ and leaves the topology as it is, whereas a learned classifier head needs a full retraining pass.

Explanation. A decision is explained by the winning node, its neighbors, the per-model error rates $\bar{e}_{sk}$, and the penalized scores that decided the arg-min (Table 2). The number of supporting training prompts n_s follows from the statistics pass, although the current implementation does not store it.

5 Experimental Setup

5.1 Data and Model Pool

Our evaluation is done on RouterBench [14]. This provides model-wise precomputed responses, quality scores and costs for 11 LLMs. These have been pooled across many benchmarking datasets and is considered a standard for evaluating new routers. We use 35,189 prompts from 86 evaluation sets from the dataset. This is because for our experimental setup, answers that do not map to any of the LLMs at all have been dropped, allowing certainty in evaluation, especially since the number dropped is very small. Every record barring undecided allocations has been used. We split them uniformly at random (seed 42) into 24,632 training prompts (70%) and 10,557 test prompts (30%). All routers share this split.

Table 3 gives each model's mean accuracy and mean cost per query on the test split. Five of the 11 models are on the accuracy-cost Pareto frontier (Fig. 2):

Table 1. Hyperparameters used for the reported results.

Method	Hyperparameter	Value
All routers	embedding model, d	bge-small, 384
GNG (both)	passes over training data T	5
GNG (both)	learning rates ϵ_b, ϵ_n	0.05, 0.005
GNG (both)	max edge age $a_{\max}$	50
GNG (both)	insertion discount α	0.5
GNG (both)	error decay γ (per step)	0.99
GNG (both)	insertion threshold θ	3.0
GNG (both)	max nodes $N_{\max}$	200
GNG (both)	max edge length ρ	0.5
GNG (both)	temporal decay κ	0
GNG (both)	node lifespan τ (steps)	20,000
GNG-shared	final nodes / edges (seed 0)	107 / 661
GNG-isolated	final nodes per model	70–200 (1,095 total)
GNG-isolated	boundary penalty β	2.0
GNG-isolated	boundary radius ρ_k	90th pct.
KNN	k, distance	50, cosine
SVM	LinearSVR, C	0.1
MLP	layers, width, lr, iters	3, 100, 10^{-3}, 100
UniRoute	clusters K	10

Table 2. Fields of the per-decision explanation record (schematic).

Field	Meaning
winning node s, $\mathcal{N}(s)$	region and adjacent regions
$\bar{e}_{sk}$ for all k	per-model error rate in region
$\bar{c}_k$ for all k	per-model cost
scores $\bar{e}_{sk} + \lambda \bar{c}_k$	why the chosen model beat the others
n_s (derivable)	number of supporting training prompts

mistral-7b-chat, WizardLM-13B, mixtral-8x7b-chat, Yi-34B-Chat, and gpt-4-1106-preview. Each of the other six is beaten by a model that is both cheaper and more accurate. Yi-34B-Chat, for example, is more accurate than claude-v1, claude-v2, and gpt-3.5-turbo and costs between 8% and 76% as much. GPT-4 is 14.0 points more accurate than the next-best frontier model (0.809 against 0.669 for Yi-34B-Chat) and costs about 17.5 times as much. Any prompt that Yi-34B answers as well as GPT-4 is a saving for a router that sends it there.

5.2 Embeddings

Prompts are embedded once with `BAAI/bge-small-en-v1.5` ($d = 384$), without dimensionality reduction. Every router uses these same embeddings.

Each prompt string is stripped of leading and trailing whitespace, and prompts stored as a string representation of a list of strings (for example a system prompt

Table 3. RouterBench model pool: mean accuracy and cost per query on the test split ($n = 10{,}557$).

Model	Accuracy	Cost ($)
gpt-4-1106-preview	0.809	3.30e-3
Yi-34B-Chat	0.669	1.89e-4
claude-v2	0.654	2.45e-3
claude-v1	0.651	2.17e-3
gpt-3.5-turbo-1106	0.638	2.47e-4
claude-instant-v1	0.622	2.36e-4
mixtral-8x7b-chat	0.567	1.37e-4
WizardLM-13B-V1.2	0.447	7.42e-5
llama-2-70b-chat	0.339	2.05e-4
mistral-7b-chat	0.323	4.62e-5
code-llama-instruct-34b-chat	0.214	1.75e-4

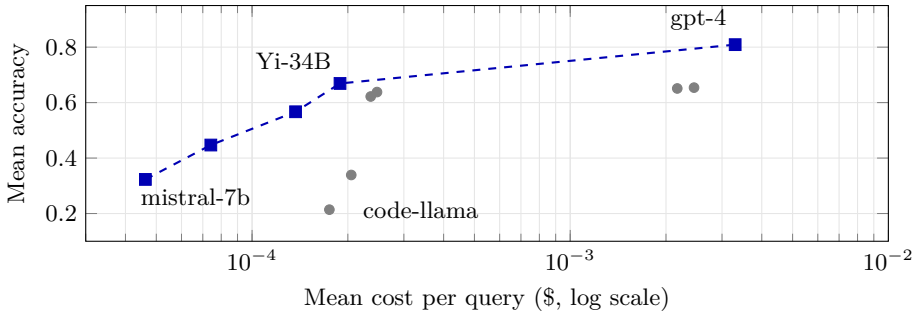


Fig. 2. The RouterBench pool (Table 3). Squares and the dashed line mark the Pareto-efficient models, and gray circles are dominated ones.

followed by a user prompt) are joined with newlines before encoding. The text is encoded in batches of 32 with the encoder's default settings through the sentence-transformers library, which truncates inputs to the encoder's maximum input length, and the embeddings are computed once and cached. The resulting vectors have unit L2 norm (over all 35,189 prompts the norms lie between 0.9999998 and 1.0000001), and no further normalization is applied. Squared Euclidean distance, as used by the GNG, therefore orders neighbors identically to the cosine distance used by the KNN router. Only this one encoder was used, so whether the routing results remain consistent across different embedding models was not evaluated.

5.3 Baselines

We compare against four routers, all trained on the same split and embeddings and swept over the same λ grid.

– *KNN*: a k-nearest-neighbor router ($k = 50$, cosine distance) that predicts each model's accuracy as its mean over the neighboring prompts [5, 21].

- *SVM*: one linear support vector regressor per model (LinearSVR, $C = 0.1$), predicting accuracy.
- *MLP*: one multilayer-perceptron regressor per model (three hidden layers of 100 ReLU units, learning rate 10^{-3}, 100 iterations), predicting accuracy.
- *UniRoute*: cluster-based universal model routing [17], with k-means ($K = 10$) over the training embeddings and per-cluster, per-model error rates.

KNN, SVM, and MLP route by $\arg\max_k \hat{q}_k(x) - \lambda \bar{c}_k$. UniRoute routes by $\arg\min_k E_{k,j} + \lambda \bar{c}_k$, where j is the cluster of the prompt. Baseline settings are fixed at the values in Table 1.

5.4 Metrics

Following UniRoute [17], we sweep λ over $\{0\} \cup \{500 \text{ log-spaced values in } [10^{-1}, 10^{6}]\}$ and obtain a deferral curve $A_r(\tilde{c})$ of accuracy against normalized cost $\tilde{c} \in [0, 1]$. Here $\tilde{c} = 0$ is the mean cost $c_{\min}$ of the cheapest model and $\tilde{c} = 1$ the mean cost $c_{\max}$ of the most expensive one. The curve is anchored at $(0, \text{accuracy of the cheapest model})$ and $(1, \text{accuracy of the most expensive model})$, and areas use trapezoidal integration over the sorted points. We report three metrics.

- *Area*: $\int_0^1 A_r(\tilde{c})\, d\tilde{c}$, the area under the deferral curve (higher is better).
- *Area50*: the same integral as Area over $\tilde{c} \in [0, 0.5]$. This looks at the concentration in the lower half of the range (higher is better).
- *QNC*: quality-neutral cost, $\min\{\tilde{c} : A_r(\tilde{c}) \geq a_{\max}\}$; the lowest normalized cost at which the router meets the accuracy $a_{\max}$ of the best single model. It is 1.0 if the router does not reach $a_{\max}$ below maximum cost (lower is better).

Here these metrics are measured on the test split. Area results are meant to describe performance over the whole range and QNC is a threshold at the high-accuracy end, important to understand if the router manages to beat the best router's cost. Percentile intervals at the 95% level from 1,000 bootstrap resamples of the test prompts have also been reported to show variability changes. The trained routers are held fixed and all routers are scored on the same resamples, so differences between routers are paired.

5.5 Implementation and Hyperparameters

Table 1 shows the hyperparameters we used in our testing. The GNG parameters are fixed defaults.

Decay convention. In the first GNG-shared run, t was the global step counter and because of $\kappa = 5 \times 10^{-4}$ the update weight fell below 1% after about 9,000 of the 123,160 training steps, which didn't allow the topology to develop further. The GNG-isolated structure measures t from the most recent step, so the weight stays at 1, a mismatch found when we compared the two systems. Since the isolated convention is also standard GNG, we decided to set $\kappa = 0$ for both of them (specifically to prevent early freezing) and retrained GNG-shared. These

Table 4. Routing performance on RouterBench (all routers on the same 384-d embeddings and split). Best values in bold. †First GNG-shared run, with update weights decayed by the global step counter (Section 5).

Router	Area ↑	Area50 ↑	QNC ↓
GNG-shared (ours)	**0.7853**	**0.3808**	**0.7214**
UniRoute	0.7818	0.3773	0.7491
KNN	0.7785	0.3760	1.0000
GNG-isolated (ours)	0.7775	0.3776	1.0000
SVM	0.7503	0.3605	1.0000
MLP	0.6815	0.3155	1.0000
GNG-shared, decayed updates†	0.7831	0.3789	0.7426

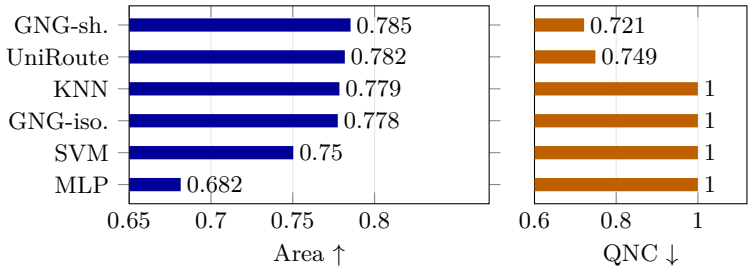


Fig. 3. Area (left, with the axis starting at 0.65 for legibility) and QNC (right, where 1.0 means GPT-4 accuracy is never matched below maximum cost).

results appear in Table 4; the decayed run is marked with a dagger. One isolated topology (code-llama) reached node limit $N_{\max} = 200$ and is densely connected (13,471 edges), because new nodes link to every node within ρ.

6 Results

Table 4 and Fig. 3 give the deferral-curve metrics on the 10,557 test prompts. All routers share $c_{\min} = 4.62 \times 10^{-5}$, $c_{\max} = 3.30 \times 10^{-3}$, and $a_{\max} = 0.809$ (GPT-4).

Overall quality. GNG-shared has the highest area (0.7853) and Area50 (0.3808). UniRoute follows (0.7818 and 0.3773), then KNN (0.7785) and GNG-isolated (0.7775). SVM (0.7503) and MLP (0.6815) are well behind, and the MLP is the weakest router, in line with the finding that simple non-parametric methods can compete with learned routers [21]. Repeating the GNG-shared training with five random seeds on the same split gives an area of 0.7856 with a standard deviation of 0.0010 (range 0.7847 to 0.7872) and 105 to 111 nodes.

Table 5 gives the bootstrap intervals. In paired comparisons GNG-shared is ahead of UniRoute by 0.0035 in area (95% CI 0.0015 to 0.0056), of KNN by 0.0068, of GNG-isolated by 0.0078, of SVM by 0.0350, and of MLP by 0.1039. None of these intervals contains zero. The advantage over UniRoute is small,

Table 5. Bootstrap results over 1,000 resamples of the test prompts †Decayed-update GNG-shared run.

Router	Area (95% CI)	Reaches $a_{\max}$
GNG-shared	0.7852 [0.7794, 0.7906]	99%
UniRoute	0.7817 [0.7760, 0.7872]	92%
KNN	0.7783 [0.7731, 0.7836]	10%
GNG-isolated	0.7774 [0.7715, 0.7827]	0%
SVM	0.7502 [0.7439, 0.7561]	0%
MLP	0.6812 [0.6748, 0.6877]	0%
GNG-shared†	0.7829 [0.7773, 0.7886]	79%

and based on only our own testing. In practice, we interpret this as GNG-shared being competitive with the strongest baseline, not as a decisive conclusive improvement by default. However, GNG does provide potential architectural advantages that should be explored further, as well as explainability which suggests better alignment with modern routing goals.

Cost to match the best model. On the test split GNG-shared reaches GPT-4's accuracy at a normalized cost of 0.721, about 28% below always calling GPT-4. UniRoute reaches it at 0.749. KNN, GNG-isolated, SVM, and MLP don't reach performance below the maximum cost, making their QNC 1.0. KNN and GNG-isolated are within 0.008 of GNG-shared in area, so a router can look close on area and still never give the accuracy of the strongest model.

QNC depends on a threshold and moves more than area does. Across the five training seeds the QNC of GNG-shared ranges from 0.567 to 0.731 (mean 0.693). Over the bootstrap resamples it reaches GPT-4's accuracy in 99% of cases, against 92% for UniRoute, 10% for KNN, and none for GNG-isolated, SVM, and MLP (Table 5). When both GNG-shared and UniRoute reach it, GNG-shared does so at a lower cost by 0.071 on average (921 resamples). A tolerance-based version of QNC, the cost to reach $a_{\max} - 0.005$, would be less sensitive to small shortfalls.

Decay convention. The decayed GNG-shared run (area 0.7831, QNC 0.7426) is 0.0022 below the earlier run in area (95% CI 0.0003 to 0.0043) and reaches GPT-4's accuracy in 79% of resamples. It remains competitive with UniRoute, so the comparison does not only hinge on this choice, but the constant-weight setting is the one we believe is the best used in general. We adopted it mainly because it is standard and matches GNG-isolated. Both runs are reported so that the effect is visible (a minimal change).

Shared versus isolated topologies. With the same update rule, GNG-shared continues to lead GNG-isolated by 0.0078 in area (95% CI 0.0052 to 0.0102), and beats GNG-isolated completely on QNC, which shows GNG-isolated never matches performance below maximum costs. We strongly believe the reason is that the boundaries blur in the case of GNG-isolated making it fall back too often to the single best model.

7 Discussion

Why regions matter when the metrics are close. On this benchmark the top routers are separated by a few thousandths of area, and we do not claim a large accuracy gain. The GNG router improves on other algorithms by the fact that each decision provided is accompanied by a stable unit to inspect. UniRoute does give one, but with ten clusters it is provides little information. The GNG-shared topology has 107 nodes for 24,632 training prompts, about 230 prompts per node, so the full table of regions and per-model error rates has only 107 rows. This means it is easy to interpret why a model goes to a cheap prompt, which regions are still dominated by the single best model and also provide insight into where each model independently needs to improve (a signal that helps develop AI). The edges also show which regions border each other, so one can see how the decision would change for a prompt near a boundary, and by how closely, which can allow algorithmic decisions on providing choices and taking user preferences in close cases. The neighbour prompts that correspond to each can be viewed and the expertise can be correlated if an audit is done.

Changing pools. The fixed-pool results show that the region-based router is competitive, and as previously mentioned, is designed to scale up (as described in Section 4.4). A potential strategy to investigate pooling is to hold out one or more models during training, and to add them later with Algorithm 2 and compare area, QNC, and prompts needed against a router retrained on the full pool. Drift in the prompt distribution calls for a similar testing which can be extensive. We leave both to future work.

Label-aware growth. The topology grows using geometric quantization error. This means there are some nodes spent where prompts are dense, even if the best model in the region doesn't change, which may lead to higher comparatives at inference. Insertion could instead follow label impurity, much as impurity guides splits in decision trees [1] but in an incremental, graph-based form.

Dimensionality and sparse regions. Nearest-neighbor distances do change in high dimensions, which can hurt geometric routing. Our paper made use of 384-dimensional embeddings without projection and did not measure this effect. We hypothise that embedding space is, by nature, semantically distributed, however further research in dimensionality effect is warranted. GNG-shared also needs enough prompts in each node for reliable per-model means, and because growth is unsupervised, a region where the best model changes sharply can fall inside a single node. Both are open questions for future work.

Deployment overhead and scalability. At inference the router computes the distance from the prompt embedding to each of the N reference vectors, takes an arg-min over the N distances, and takes an arg-min over K cost-adjusted scores (Section 4). For GNG-shared with $N = 107$, $d = 384$, and $K = 11$, the stored model consists of 107×384 reference values and 107×11 error rates, about 330 KB in double precision, whereas a KNN router must keep all 24,632 training embeddings. GNG-isolated stores 1,095 nodes across eleven topologies and performs eleven nearest-node searches per prompt. Routing latency was not measured in this paper. The cost of encoding the prompt is incurred by every

embedding-based router, including all baselines here, and is not included in this discussion. Training scales with the number of nodes and the size of the training set as described under Complexity in Section 4, and the number of nodes is capped by $N_{\max}$. The parameters were not tuned for larger pools, so scalability to pools much larger than 11 models, or to much larger prompt collections, was not evaluated.

Societal impact. Routing each prompt to the cheapest model likely to answer it correctly can lower the cost of using language models, which matters most for organizations with limited budgets, and the stored region statistics let operators audit where cheaper models suffice. We did not measure energy use, so we do not quantify any environmental benefit.

8 Limitations and Threats to Validity

The evaluation uses RouterBench and one random 70/30 split (seed 42). Given limited compute and budget, we did not repeat the full pipeline across several splits. We did vary the GNG-shared training seed (five seeds) and used bootstrap intervals over test prompts for sampling variation, but the intervals do not capture variation across splits, and GNG-isolated was not bootstrapped or reseeded. Repeating the study on several splits with a held-out validation set is the natural next step. A validation-based selection of β is planned for a potential multi-split study.

The 70/30 split is random, so the test prompts come from the same 86 evaluation sets as the training prompts. Generalization to unseen prompt categories and to distribution shift was therefore not evaluated, and neither was the addition of a new LLM without retraining, which Algorithm 2 describes as a design only. The routing results were obtained with a single embedding model, and their consistency across embedding models was not evaluated. Prices are a fixed snapshot in RouterBench, so the sensitivity of the results to changes in model prices was not evaluated either.

RouterBench is dominated by a few sources: the largest evaluation set (hellaswag) accounts for 27.8% of the 35,189 prompts, grade-school-math for 21.2%, and the five largest sets together for 60.8%, while the median evaluation set has 140 prompts and 20 sets have fewer than 50. The aggregate metrics therefore mostly reflect performance on a few benchmarks, and region-level statistics for rarely represented task types rest on few prompts. The margin over UniRoute is small, and UniRoute was run with $K = 10$ clusters whereas GNG-shared uses 107 nodes, so the comparison does not separate the effect of the number of regions from the effect of the GNG topology.

9 Reproducibility and Ethical Considerations

All inputs are the public RouterBench data [14], and the routers use only embeddings and stored per-model scores due to their sparse design. Table 1 and Algorithm 1 specify the method. The split is fixed by seed 42 and the

baseline seeds are fixed in the code. The GNG-shared result in Table 4 uses NumPy seed 0 for the presentation order, and seeds 1 to 4 are reported in the text. A miscalibrated router can send some kinds of prompts to weaker models systematically. Region-level statistics make such patterns visible, but we did not assess fairness across user groups or languages.

10 Conclusion and Future Work

Our paper provides the architecture for a region based heterogeneous router for a pool of LLMs, building on the range of routers being built in this field. A GNG approach, based on error rates stored in notes and a cost aware parameter helps keep it competitive with other state of the art routers in the field with potential advantages. Considering RouterBench performance, the strategy is competitive with UniRoute, KNN, SVM, and MLP. The area is 0.0035 above UniRoute's. We regard the router as a method that matches strong baselines while making its decisions inspectable. The design was built with explainable routing and evolving pools as primary goals, and evaluating that the latter is the main item for future work. Evaluating on LLMRouterBench [20], projection to reduce dimensionality effects, and compiling the trained topology into a hash-based lookup (for example locality-sensitive hashing [15], in the spirit of hash routing [28]) for near-constant-time inference are also avenues to explore.

References

1. Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: Classification and Regression Trees. Wadsworth (1984)
2. Cao, H., Wang, Y., Li, T., Yao, X., Zhu, P.: Geometric mixture-of-experts with curvature-guided adaptive routing for graph representation learning. arXiv:2603.22317v1 (2026)
3. Carpenter, G.A., Grossberg, S.: A massively parallel architecture for a self-organizing neural pattern recognition machine. Computer Vision, Graphics, and Image Processing **37**, 54–115 (1987)
4. Chen, L., Zaharia, M., Zou, J.: FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176 (2023)
5. Cover, T., Hart, P.: Nearest neighbor pattern classification. IEEE Trans. Information Theory **13**(1), 21–27 (1967)
6. Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Rühle, V., Lakshmanan, L.V.S., Awadallah, A.H.: Hybrid LLM: Cost-efficient and quality-aware query routing. In: Proc. Int. Conf. Learning Representations (ICLR) (2024)
7. Fedus, W., Zoph, B., Shazeer, N.: Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. J. Machine Learning Research **23**(120), 1–39 (2022)
8. Feng, T., Shen, Y., You, J.: GraphRouter: A graph-based router for LLM selections. In: Proc. Int. Conf. Learning Representations (ICLR) (2025)
9. Fritzke, B.: A growing neural gas network learns topologies. In: Advances in Neural Information Processing Systems 7. pp. 625–632 (1995)

10. Fritzke, B.: A self-organizing network that can follow non-stationary distributions. In: Artificial Neural Networks (Proc. ICANN). pp. 613–618 (1997)
11. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Computing Surveys **46**(4), 44:1–44:37 (2014)
12. Gururangan, S., Li, M., Lewis, M., Shi, W., Althoff, T., Smith, N.A., Zettlemoyer, L.: Scaling expert language models with unsupervised domain discovery. arXiv:2303.14177v1 (2023)
13. He, N., Anand, R., Madhu, H., Maatouk, A., Krishnaswamy, S., Tassiulas, L., Yang, M., Ying, R.: HELM: Hyperbolic large language models via mixture-of-curvature experts. In: Proc. 39th Conf. Neural Information Processing Systems (NeurIPS) (2025)
14. Hu, Q.J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., Upadhyay, S.K.: RouterBench: A benchmark for multi-LLM routing system. arXiv:2403.12031v2 (2024)
15. Indyk, P., Motwani, R.: Approximate nearest neighbors: Towards removing the curse of dimensionality. In: Proc. 30th ACM Symp. Theory of Computing (STOC). pp. 604–613 (1998)
16. Jacobs, R.A., Jordan, M.I., Nowlan, S.J., Hinton, G.E.: Adaptive mixtures of local experts. Neural Computation **3**(1), 79–87 (1991)
17. Jitkrittum, W., Juneja, J., Go, A., Panigrahy, R., Narasimhan, H., Wang, C., Lee, C.Y., Menon, A.K., Rawat, A.S., Wang, Z., Shenoy, P., Kumar, S.: Universal model routing for efficient LLM inference. arXiv:2502.08773v2 (2025)
18. Kohonen, T.: Self-organized formation of topologically correct feature maps. Biological Cybernetics **43**(1), 59–69 (1982)
19. Lail, R., Markham, L.: On cost-effective LLM-as-a-judge improvement techniques. arXiv:2604.13717v3 (2026)
20. Li, H., Zhang, Y., Guo, Z., Wang, C., Tang, S., Zhang, Q., Chen, Y., Qi, B., Ye, P., Bai, L., Wang, Z., Hu, S.: LLMRouterBench: A massive benchmark and unified framework for LLM routing. arXiv:2601.07206v1 (2026)
21. Li, Y.: Rethinking predictive LLM routing: When simple KNN beats complex learned routers. arXiv:2505.12601v2 (2026)
22. Martinetz, T., Schulten, K.: A "neural-gas" network learns topologies. In: Artificial Neural Networks (Proc. ICANN). pp. 397–402 (1991)
23. Martinetz, T., Schulten, K.: Topology representing networks. Neural Networks **7**(3), 507–522 (1994)
24. Moslem, Y., Kelleher, J.D.: Dynamic model routing and cascading for efficient LLM inference: A survey. arXiv:2603.04445v2 (2026)
25. Mu, S., Lin, S.: A comprehensive survey of mixture-of-experts: Algorithms, theory, and applications. arXiv:2503.07137v4 (2026)
26. Ong, I., Almahairi, A., Wu, V., Chiang, W.L., Wu, T., Gonzalez, J.E., Kadous, M.W., Stoica, I.: RouteLLM: Learning to route LLMs with preference data. In: Proc. Int. Conf. Learning Representations (ICLR) (2025)
27. Pichlmeier, J., Ross, P., Luckow, A.: Performance characterization of expert router for scalable LLM inference. arXiv:2404.15153v2 (2024)
28. Roller, S., Sukhbaatar, S., Szlam, A., Weston, J.: Hash layers for large sparse models. arXiv:2106.04426v3 (2021)
29. Rudin, C.: Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence **1**, 206–215 (2019)

30. Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., Dean, J.: Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In: Proc. Int. Conf. Learning Representations (ICLR) (2017)
31. Shirkavand, R., Yu, P., Gao, S., Huang, H.: Cost-aware contrastive routing for LLMs. In: Proc. 39th Conf. Neural Information Processing Systems (NeurIPS) (2025)
32. Song, W., Huang, Z., Cheng, C., Gao, W., Xu, B., Zhao, G., Wang, F., Wu, R.: IRT-Router: Effective and interpretable multi-LLM routing via item response theory. arXiv:2506.01048v2 (2025)
33. Ternovtsii, I., Bilak, Y.: Equifinality in mixture of experts: Routing topology does not determine language modeling quality. arXiv:2604.14419v1 (2026)
34. Varangot-Reille, C., Bouvard, C., Gourru, A., Ciancone, M., Schaeffer, M., Jacquenet, F.: Doing more with less – implementing routing strategies in large language model-based systems: An extended survey. arXiv:2502.00409v2 (2025)
35. Varshney, T., Surla, A., Xu, M., Krishnan, G.V., Jeblick, M., Austin, D., Vaidya, N., Onofrio, D.: LLM router: Rethinking routing with prefill activations. arXiv:2603.20895v2 (2026)
36. Wang, C., Liu, X., Liu, Y., Zhu, Y., Mo, X., Jiang, J., Chen, H.: When to reason: Semantic router for vLLM. arXiv:2510.08731 (2025)
37. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. IEEE Trans. Evolutionary Computation **1**(1), 67–82 (1997)
38. Zhang, H., Feng, T., You, J.: Router-R1: Teaching LLMs multi-round routing and aggregation via reinforcement learning. arXiv:2506.09033v3 (2025)
39. Zhang, W., Zhang, L., Xiang, L., Cai, H.: Reasoning is not free: Robust adaptive cost-efficient routing for LLM-as-a-judge. arXiv:2605.10805 (2026)
40. Zhuang, R., Wu, T., Wen, Z., Li, A., Jiao, J., Ramchandran, K.: EmbedLLM: Learning compact representations of large language models. arXiv:2410.02223v2 (2024)